



Python Fundamentos

Introducción

1



Una función se presenta como un bloque de código con un nombre específico que encapsula un conjunto de operaciones. Esta estructura puede tomar parámetros de entrada y devolver resultados y parámetros de salida, siendo invocada desde diferentes partes de un programa para llevar a cabo una tarea específica.

También nos reportan una serie de ventajas a la hora de crear y diseñar código:

- Permiten reutilizar el código contenido en las mismas sin tener que duplicarlo en otras partes.
- Facilita el mantenimiento y la legibilidad del código.
- Ya que es posible declarar funciones para cualquier tarea, podemos dividir el código en pequeñas encapsulaciones funcionales que dividan problemas más grandes en subproblemas más pequeños.
- Es posible ejecutar una función de otro autor sin tener pleno conocimiento de su funcionamiento interno.
- Dada su funcionalidad constante, facilita la búsqueda de errores e incidencias en un bloque de código más grande.
- Permite el reparto de tareas y subtareas entre programadores de un mismo equipo.



A la hora de definir una función encontramos el encabezado y el cuerpo.

Encabezado	{	<code>def area_circulo(r):</code>
		<code>'''Función que recibe el radio del círculo y calcula su área'''</code>
Cuerpo	{	<code>area = 3.1415*r**2</code>
		<code>return area</code>

La utilidad del encabezado es la de dar nombre a la función y recoger los parámetros entre paréntesis necesarios para su correcta ejecución (aunque es posible que su ejecución no necesite de ningún parámetro de entrada). La palabra 'def' se utiliza específicamente para a continuación declarar el nombre que identificará a la función.

Por otro lado tenemos el cuerpo de la función, que ha de tener una indentación distinta al encabezado para que Python pueda identificarlo. Las variables utilizadas en él, como es en nuestro caso el parámetro/variable 'r', se comportan como variables locales que sólo estarán accesibles dentro del cuerpo de la función. Cuando definimos una función, el código contenido dentro del cuerpo no se ejecuta inmediatamente. Este únicamente será ejecutado cuando realicemos la llamada a la misma.

Cabe destacar que al igual que la palabra 'def', la palabra 'return' es necesaria para definir una función si deseamos devolver una o varias variables al final de la misma.



Si una función dispone de más de un parámetro es necesario seguir una reglas a la hora de hacer la llamada. Así, es posible llamar a una función colocando en el orden correcto los parámetros que necesita, o nombrandolos a la hora de incluirlos en la misma.

python

```
def potencia(base, exponente):  
    resultado = base ** exponente  
    return resultado
```

En el ejemplo que vemos en la imagen superior encontramos a la función 'potencia', cuyos argumentos son 'base' y 'exponente'. Si queremos calcular el resultado de 2^3 , lo podremos hacer de las siguientes formas:

- `potencia(2, 3)`: En este caso Python entiende que el número 2 se corresponde con el primer argumento 'base', mientras que el número 3 se corresponde con el argumento 'exponente'.
- `potencia(base = 2, exponente = 3)`: Aquí sí que estamos especificando claramente que el valor del parámetro 'base' ha de ser 2 y del 'exponente' será 3.



También es posible declarar argumentos por defecto dentro de una función. Estos tienen la capacidad de seleccionar un valor constante, siempre y cuando la llamada a la función no sobreescriba dicho valor. Tomando el ejemplo de la diapositiva anterior, si un gran porcentaje de llamadas van a requerir que el parámetro 'exponente' sea igual a 2, podemos declarar este parámetro con dicho valor por defecto. Si no es necesaria ninguna corrección de dicho parámetro, únicamente habrá de especificarse 'base'. Por el contrario, incluir el valor deseado en el parámetro correcto es necesario de cara a poder superar la designación por defecto.

```
python

def potencia(base, exponente=2):
    resultado = base ** exponente
    return resultado

resultado_potencia = potencia(3)
print(f"3 elevado al cuadrado es {resultado_potencia}")

resultado_potencia_cubica = potencia(3, 3)
print(f"3 elevado al cubo es {resultado_potencia_cubica}")
```



En Python, es posible crear funciones que acepten un número indeterminado de parámetros. Esto es especialmente útil cuando no se sabe de antemano cuántos argumentos se pasarán a la función. Python ofrece dos maneras principales de lograr esto: usando `*args` para argumentos posicionales y `**kwargs` para argumentos nombrados (keyword arguments).

1. `*args`: Se usa para recibir un número indeterminado de argumentos posicionales. Los argumentos se almacenan en una tupla.
1. `**kwargs`: Se usa para recibir un número indeterminado de argumentos nombrados. Los argumentos se almacenan en un diccionario.

```
def sumar_todos(*args):  
    """Suma todos los números que se pasan como argumentos."""  
    return sum(args)  
  
# Ejemplo de uso  
resultado = sumar_todos(1, 2, 3, 4, 5)  
print(f"La suma es: {resultado}") # La suma es: 15
```

```
def crear_diccionario(**kwargs):  
    """Crea un diccionario a partir de los argumentos nombrados que se pasan."""  
    return kwargs  
  
# Ejemplo de uso  
resultado = crear_diccionario(nombre="Juan", edad=30, ciudad="Madrid")  
print(f"El diccionario es: {resultado}") # El diccionario es: {'nombre': 'Juan', 'edad': 30, 'ciudad': 'Madrid'}
```

Recursividad

2



Hablamos de algoritmos recursivos cuando una posible dirección de ejecución es una nueva llamada a sí mismo.

Ha de cumplir dos cuestiones principales:

- Para el algoritmo ha de existir al menos una solución mínima para evitar así la ejecución infinita del mismo.
- A medida que se realizan llamadas sucesivas, la complejidad del problema ha de reducirse.

Vemos en el siguiente código que la función 'factorial' se llama a sí misma con un valor decrementado tras cada paso ($n - 1$) hasta que alcanza el caso base ($n == 0$ o $n == 1$). En ese momento la recursividad se detiene y la función devuelve 1. La multiplicación de todos los valores regresados en las llamadas recursivas da como resultado el factorial del número: $5! = 5 * 4 * 3 * 2 * 1$, que es igual a 120.

```
python

def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

# Ejemplo de llamada a la función recursiva
resultado_factorial = factorial(5)
print(f"El factorial de 5 es {resultado_factorial}")
```

Lambdas

3



Las lambdas en Python, también conocidas como funciones anónimas, son una característica poderosa y versátil del lenguaje.

Un lambda es una pequeña función anónima, que puede tener cualquier número de argumentos pero solo una expresión. Son útiles para crear funciones cortas y desechables sin necesidad de definirlos formalmente utilizando 'def' como veremos en el apartado de funciones.

```
python  
  
lambda argumentos: expresión
```

A continuación explicamos la sintaxis:

- 'lambda': Palabra clave para definir una función lambda
- 'argumentos': Una lista de argumentos separados por comas, similar a los argumentos de una función
- 'expresión': Una sola expresión que se evalúa y devuelve. No se permiten múltiples expresiones o declaraciones



Un ejemplo básico de una función lambda que suma dos números

```
python

suma = lambda x, y: x + y
resultado = suma(2, 3)
print(resultado) # 5
```

Como vemos en la imagen superior, definimos la función lambda que guardamos en la variable suma. Como argumentos se definen 'x' e 'y' y como expresión (lo que va a realizar la función lambda) que sume los valores de 'x' e 'y'.

Luego en la variable resultado guardamos el resultado que devuelve la función lambda pasando los valores de 2 y 3 por lo que el resultado obtenido es 5.

Este es un ejemplo básico del funcionamiento de un lambda que nos demuestra lo útil y versátil que pueden llegar a ser.



Uno de los usos más comunes de los lambdas es como argumentos para funciones de orden superior, que veremos más adelante.

- `map()`: Aplica una función a todos los elementos de un iterable

```
python

numeros = [1, 2, 3, 4]
cuadrados = map(lambda x: x**2, numeros)
print(list(cuadrados)) # [1, 4, 9, 16]
```

- `filter()`: Filtra los elementos de un iterable que cumplen con una condición

```
python

numeros = [1, 2, 3, 4]
pares = filter(lambda x: x % 2 == 0, numeros)
print(list(pares)) # [2, 4]
```

- `sorted()`: Ordena un iterable utilizando una función clave

```
estudiantes = [("Ana", 21), ("Juan", 22), ("María", 20)]
ordenado_por_edad = sorted(estudiantes, key=lambda estudiante: estudiante[1])
print(ordenado_por_edad) # [("María", 20), ("Ana", 21), ("Juan", 22)]
```



Otro de los posibles usos de los lambdas es para funciones cortas que se definen en una sola línea.

```
python
```

```
# Ejemplo de una lambda que convierte grados Celsius a Fahrenheit  
celsius_a_fahrenheit = lambda c: (c * 9/5) + 32  
print(celsius_a_fahrenheit(25)) # 77.0
```

Ventajas de los lambdas:

- Simplicidad: Los lambdas son concisos y pueden hacer el código más legible
- Función inline: Perfectos para funciones pequeñas que se utilizan solo una vez, evitando la necesidad de definir una función completa

Limitaciones:

- Inmutabilidad: Los lambdas están limitados a una sola expresión y no pueden contener múltiples declaraciones o expresiones complejas
- Legibilidad: Un uso excesivo de lambdas puede hacer que el código sea más difícil de entender, especialmente para funciones más largas o complejas

Funciones de orden superior

4



Definimos una función de orden superior si esta devuelve una función como parámetro de salida o contiene alguna función como parámetro de entrada.

Tenemos a la función 'filter', que se utiliza para filtrar elementos de una secuencia según una condición. Utilizando una función 'lambda', podemos extraer de una lista de números sólo aquellos que sean números pares.

```
python

# Lista de números
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Usar filter con una función lambda para obtener los números pares
numeros_pares = list(filter(lambda x: x % 2 == 0, numeros))

print(numeros_pares)
```



La función 'map' no permite aplicar una función sobre cada uno de los elementos de una secuencia. Por ejemplo, tenemos una lista y queremos elevar al cuadrado cada uno de los elementos de la misma. Devuelve un objeto del tipo 'map()', con lo que tendremos que aplicar la función list() para convertir el resultado en una lista fácilmente legible.

```
python

# Lista de números
numeros = [1, 2, 3, 4, 5]

# Usar map con una función lambda para elevar al cuadrado
cuadrados = list(map(lambda x: x**2, numeros))

print(cuadrados)
```

La función map() es útil cuando se desea aplicar una operación a cada elemento de una secuencia y obtener los resultados en una nueva secuencia.



Según una función que aplica de izquierda a derecha sobre un iterable objetivo, reduce el conjunto a un sólo valor. Lleva a cabo un proceso de agregación de los datos del conjunto. Es una función que se encuentra en el módulo 'functools', por lo que ha de ser importada previamente.

A diferencia de 'map' y 'filter' tiene un argumento extra opcional denominado 'valor_inicial'. Si se especifica, comienza a aplicar la función acumulativa primero a este valor y continuando con el primer valor del iterable objetivo.

En la imagen vemos como a 'reduce' se le ha pasado como argumento una función lambda que hará de función acumulativa de los sucesivos valores de la lista números. El resultado asignado a la variable 'suma_total' será 15.

```
python

from functools import reduce

# Lista de números
numeros = [1, 2, 3, 4, 5]

# Usar reduce con una función lambda para sumar los elementos
suma_total = reduce(lambda x, y: x + y, numeros)

print(suma_total)
```



También podemos crear nuestras propias funciones de orden superior que al igual que las predefinidas son aquellas que pueden aceptar otras funciones como argumentos, retornar una función como resultado, o ambas cosas. Este concepto es una característica fundamental en muchos lenguajes de programación funcional y permite escribir código más modular y reutilizable.

1. Son funciones que aceptan otras funciones como argumentos: Estas funciones pueden tomar una o más funciones como entrada y operar con ellas.
1. Son funciones que retornan otras funciones: Estas funciones pueden crear y devolver nuevas funciones.

Definimos las funciones

```
def aplicar_operacion(numeros, operacion):  
    """Aplica una operación a cada elemento de una lista de números."""  
    return [operacion(numero) for numero in numeros]  
  
# Ejemplo de función que se puede pasar como argumento  
def al_cuadrado(x):  
    return x * x  
  
def al_cubo(x):  
    return x * x * x
```

Utilizamos las funciones

```
# Lista de números  
numeros = [1, 2, 3, 4, 5]  
  
# Aplicar la operación al cuadrado  
resultados_cuadrado = aplicar_operacion(numeros, al_cuadrado)  
print(f"Al cuadrado: {resultados_cuadrado}") # Salida: [1, 4, 9, 16, 25]  
  
# Aplicar la operación al cubo  
resultados_cubo = aplicar_operacion(numeros, al_cubo)  
print(f"Al cubo: {resultados_cubo}") # Salida: [1, 8, 27, 64, 125]
```

Funciones del sistema

5



En el mundo de la programación, la eficiencia y la reutilización del código son fundamentales. Python ofrece una amplia gama de funciones del sistema que nos permiten interactuar con el entorno operativo, gestionar archivos, ejecutar comandos y mucho más. Estas funciones son esenciales para realizar tareas comunes y a menudo repetitivas, simplificando el desarrollo y aumentando la productividad. El intérprete de Python tiene una serie de funciones y tipos incluidos en él que están siempre disponibles.

Funciones del sistema más comunes:

1. **open()**: Abre un archivo y devuelve un objeto de archivo.
2. **read()**: Lee el contenido de un archivo.
3. **write()**: Escribe datos en un archivo.
4. **print()**: Muestra información en la consola.
5. **input()**: Recibe entrada del usuario.
6. **type()**: Devuelve el tipo de una variable o valor.
7. **len()**: Devuelve la longitud de un objeto (como una lista, cadena, etc.).
8. **int(), str(), float()**: Convertidores de tipo de datos.



python

```
# Crear y escribir en un archivo
with open('archivo.txt', 'w') as archivo:
    archivo.write('Hola, este es un archivo de prueba.')
```

```
# Leer el contenido del archivo
with open('archivo.txt', 'r') as archivo:
    contenido = archivo.read()
print(f'Contenido del archivo: {contenido}')
```

```
# Obtener la longitud del contenido
longitud = len(contenido)
print(f'Longitud del contenido: {longitud}')
```

```
# Convertir la longitud a una cadena
longitud_str = str(longitud)
print(f'Longitud como cadena: {longitud_str}')
```

En este ejemplo:

1. Se crea un archivo y se escribe en él.
2. Se lee el contenido del archivo y se muestra en la consola.
3. Se calcula y muestra la longitud del contenido.
4. Se convierte la longitud a una cadena y se muestra.



Built-in Functions				
<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

Generadores

6



Python

```
import csv
def leer_csv (nombredefichero):
    with open(nombredefichero, 'r') as fichero:
        tmp = csv.reader(fichero)
        for línea in tmp:
            yield línea
for línea in leer_csv ('test.csv'):
    print(línea)
```

Son funciones especiales en Python que permiten producir una secuencia de valores de manera eficiente que utiliza la palabra clave 'yield' en lugar de 'return'. Optimizando el uso de memoria y rendimiento en Python. Un generador devuelve un iterador sin almacenar todos los valores en memoria

¡Gracias!